Web Application with Hibernate (using XML)

Example 1: (Need to be checked)

To create a web application with Hibernate using XML configuration, you'll need to set up a Java web project and configure Hibernate to work with your web application. Here are the steps:

Step 1: Set up the Project
1. Create a new Java web project in your preferred IDE (Eclipse, IntelliJ, etc.).
2. Add the required libraries to your project's classpath, including Hibernate and the database driver.

Step 2: Configure Hibernate
1. Create a Hibernate configuration file (e.g., `hibernate.cfg.xml`) in the project's classpath.
2. Configure the database connection details, dialect, and other properties in the configuration file. For example:

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <!-- Database connection settings -->
        <property name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/mydatabase</property>
        <property name="hibernate.connection.username">your_username</property>
        <property name="hibernate.connection.password">your_password</property>

        <!-- Hibernate dialect for the corresponding database -->
        <property name="hibernate.dialect">org.hibernate.dialect.MySQL5Dialect</property>

        <!-- Enable Hibernate's automatic session context management -->
        <property name="current_session_context_class">thread</property>

        <!-- Enable the Hibernate SQL logs for debugging (optional) -->
        <property name="hibernate.show_sql">true</property>
```

```
        <!-- Mapping metadata -->
        <mapping resource="com/yourpackage/Student.hbm.xml"/>
    </session-factory>
</hibernate-configuration>
```

Step 3: Define the Mapping File
1. Create a Hibernate mapping file (e.g., `Student.hbm.xml`) in your project's classpath.
2. Define the mappings between the Java entity and the database table using XML. Include the class, id, and property elements as appropriate.

Here's an example of the `Student.hbm.xml` file:

Student.hbm.xml

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
        "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="com.yourpackage.Student" table="students">
        <id name="id" column="id">
            <generator class="native"/>
        </id>
        <property name="name" column="name"/>
        <property name="age" column="age"/>
    </class>
</hibernate-mapping>
```

Step 4: Create the Entity Class
1. Create a Java class for your entity (e.g., `Student`) in your project's package (e.g., `com.yourpackage`).
2. Define the necessary fields, getters, and setters.

Here's an example of the `Student` class:

Student.java

```
public class Student {
    private int id;
    private String name;
    private int age;
```

```
    // Constructors, getters, and setters
}
```

Step 5: Perform Database Operations
1. In your servlet or other suitable web component, initialize the Hibernate SessionFactory.
2. Use the Session object to perform CRUD operations or execute queries.

Here's an example of a servlet that saves a student to the database:

StudentServlet.java

```
import javax.servlet.*;
import javax.servlet.http.*;
import org.hibernate.*;
import com.yourpackage.Student;

public class StudentServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    private SessionFactory sessionFactory;

    @Override
    public void init() throws ServletException {
        sessionFactory = HibernateUtil.getSessionFactory();
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        Session session = null;
        try {
            session = sessionFactory.getCurrentSession();
            session.beginTransaction();

            // Create a new student
            Student student = new Student();
            student.setName(request.getParameter("name"));
            student.setAge(Integer.parseInt(request.getParameter("age")));

            // Save the student to the database
```

```
            session.save(student);

            session.getTransaction().commit();
            response.getWriter().println("Student saved successfully!");
        } catch (Exception e) {
            e.printStackTrace();
            session.getTransaction().rollback();
        } finally {
            if (session != null && session.isOpen()) {
                session.close();
            }
        }
    }

    @Override
    public void destroy() {
        if (sessionFactory != null) {
            sessionFactory.close();
        }
    }
}
```

Make sure to update the appropriate servlet mappings in your web.xml file.

That's it! You've created a basic web application using Hibernate with XML configuration. You can expand on this example by adding more entities, performing additional database operations, and exploring Hibernate's features like querying and relationships. Remember to handle exceptions appropriately and close the resources to ensure proper cleanup.